

Integración llama.cpp · Asistente «Wilmercito» en Soporte_Tecnico2

Rama: dev_llama.cpp · Última actualización: 2026-09-25 Esta rama contiene **solo** el stack Python (backend-fastapi/ + frontend-django/) más la integración IA. El stack legacy V1 (.NET backend/, Next.js frontend/, mail-service/, docker-compose*.yaml, Soporte_Tecnico.sln, .env SMTP) se eliminó aquí (commit 1ed663e); sigue intacto en dev.

1. Objetivo

Integrar un asistente conversacional (**Wilmercito**) dentro del software existente Soporte_Tecnico2, reutilizando la idea validada en el proyecto donante Adlux-Connect (RAG + restricciones), pero con **llama.cpp directo en GPU** en lugar de Ollama.

Decisión del proyecto **Ollama no se usa**. Si llama.cpp fallara, el fallback sería Ollama, pero mientras tanto el servicio ollama.service queda detenido y deshabilitado en el host y no forma parte del diseño.

2. Arquitectura

```
.. HOST server-mattias (100.78.144.4) .....
. SOLO hardware + llama.cpp .
.   . RTX 2060 6 GB .
.   . ~/llama.cpp (compilado GGML_CUDA, arch sm_75) .
.   . ~/modelos/qwen2.5-1.5b-instruct-q4_k_m.gguf .
.   . llama-server :8081 (API OpenAI-compatible, con api-key) .
.   ufw: solo 100.90.209.98 .
.....
.   . Tailscale .
.. VM upds (100.90.209.98) .....
. TODO el software (demo final aquí) .
.   . ~/Soporte_Tecnico2 (copia por scp, sin .git) .
.   . PostgreSQL 15 :5432 · DBs `soporte`, `soporte_dev` .
.   . backend-fastapi :5012 (uvicorn) .
.   . frontend-django :8011 (runserver) .
.   . Chroma + embeddings CPU (colección `soporte`) .
.   . endpoints /ia/* · retrieval local + POST al .
.   llama-server del host .
.....
```

Flujo de una pregunta: burbuja Wilmercito (Django) · POST /ia/preguntar (FastAPI) · embedding + top-k Chroma (VM) · prompt canónico + contexto · POST host:8080/v1/chat/completions · respuesta con fuente (id de atención).

3. Inventario verificado

| Recurso | Valor | |---|---| | Host GPU | RTX 2060 6144 MiB, CUDA 12.4 (nvcc), 16 cores | | Modelo | Qwen2.5-3B-Instruct Q4_K_M (2.0 GB) · mejor español que llama3.2:3b, cabe en 6 GB VRAM | | VM | Debian 12, 5.8 GB RAM, 27 GB libres, Python 3.11.2, PG 15.19 | | DB soporte (VM) | 313 atenciones, 10 usuarios (copia live) | | DB soporte_dev (VM) | 2771 atenciones, 12 usuarios, 56 áreas | | Django en VM | 4.2.16 (el pin django==6.1.1 no existe en PyPI · corregido commit 8bd7be5) | | API VM | GET :5012/health · {"status":"ok", "db":"up"} | | Web VM | GET :8011/ · 302 (redirect a login, esperado) |

4. Procedimiento reproducido (bitácora real)

- 1 **Rama limpia:** `git checkout -b dev_llama.cpp` desde dev; `git rm` del legacy; fix `.gitignore` para symlinks `.venv` (el / final no ignora symlinks).
- 2 **SSH sin password:** `ssh-copy-id a mattias@100.90.209.98` (upds tiene sudo sin password).
- 3 **PostgreSQL en VM:** `apt install postgresql; CREATE ROLE soporte, DBs soporte/soporte_dev.`
- 4 **Backup:** `pg_dump -Fc local (PG16)`. El restore en PG15 falló (`unsupported version 1.15`) · se re-volcó **SQL plano** (-Fp) y se restauró con `psql`. Conteos verificados iguales al origen. Dumps en `backups/` (ignorado por git).
- 5 **Código a la VM por scp (no git):** `tar` excluyendo `.venv`, `__pycache__`, `.git`, `logs/`, `backups/`, CSVs grandes. `.env` incluidos; en la VM se corrigió `DATABASE_URL` de `:5433` · `:5432` (PG local de la VM).
- 6 **venvs en VM:** `python3 -m venv + pip install -r requirements.txt` en ambos proyectos.
- 7 **llama.cpp en host:** `git clone, cmake -DGGML_CUDA=ON -DCMAKE_CUDA_ARCHITECTURES=75, cmake --build --target llama-server llama-cli.`
- 8 **Ollama fuera:** `systemctl stop + disable ollama` (binario conservado; desinstalar queda a decisión).
- 9 **Acceso por Tailscale:** la VM no tiene ufw (pasa directo); se agregó `100.90.209.98` a `DJANGO_ALLOWED_HOSTS` del `.env` de la VM y se reinició `soporte-web-ia`. Verificación desde el host: `curl http://100.90.209.98:8011/ · 302 + login con CSRF.`

5b. Loop de aprendizaje (calificar · curar · promover)

Sin Django-admin: el equivalente vive en la burbuja + vista Conocimiento.

- 1 Cada respuesta muestra `·POST /wilmercito/calificar/ ·POST /api/ia/calificar` (tabla `FeedbackIA`, migración `0008`).
- 2 El jefe abre `/conocimiento/` (solo `_puede_dashboard`) y ve pendientes (puntaje · 3).
- 3 **Promover** `·POST /api/ia/feedback/{id}/promover` indexa `feedback_<id>` en Chroma; la próxima pregunta similar lo recupera con esa fuente. Verificado e2e.

5c. Todo el sistema como conocimiento (2844 docs)

El índice ya no es solo tickets `indexar()` cubre Atenciones (ficha completa: descripción + solución + categoría + área + medio + observaciones), Usuarios activos (nombre/rol/especialidad), Áreas (con su grupo) y feedback promovido. Además hay intención SQL determinista (`ESTADISTICAS`: totales, mes actual, top categorías) Fuentes legibles para técnicos: Base de conocimiento, Atención #N, Personal del sistema, Organización, Datos del sistema, Conocimiento del equipo.

5d. Tools estilo MCP (app/services/ia_tools.py)

Registro de tools con schemas JSON compatibles MCP (`buscar_atenciones`, `guiar_creacion`): el router de intenciones las ejecuta localmente. Decisión consciente: con un 1.5B el function-calling del modelo no es fiable para demo; los intents deterministas + confirmación humana dan 100% y el paso a un servidor MCP real es exponer este mismo registro. Poderes futuros (crear borrador con confirmación) siguen este camino, siempre con el JWT del usuario y sus roles.

5e. Mejoras Python sobre el LLM (modelo de 2 GB rindiendo como grande)

- **Reranking cross-encoder** (mmarco-mMiniLMv2, multilingüe, CPU en VM) 2da etapa sobre 12 candidatos · top-3 por relevancia real, no solo distancia **Hallazgo medido 2026-09-25: APAGADO por defecto (RERANK=1 lo activa)**. El reranker ordenó la KB exacta 6ta de 12 en "cuáles son las categorías mientras el bi-encoder la pone 1ra (0.152). En este dominio el bi-encoder solo gana; el código queda para el informe y futuros modelos.
- **Intents semánticos, no listas de palabras** (clasificar ()): el mismo embedding compara la pregunta contra ~5 ejemplos por intent (saludo, identidad, crear, capacidad, estadísticas). Batería medida: intents 0.75·1.00, resto 0.45. Umbral 0.55 + margen 0.08. El jailbreak y los IDs de ticket siguen siendo regex (seguridad y extracción exacta: ahí el código determinista es lo correcto).
- **Híbrido vectorial + BM25 (RRF)** keywords exactas (IDs, códigos) + semántica. El modelo a veces ecoaba el id del contexto ([atencion_N]): formato movido al final + post-chequeo que cae al texto curado si responde un id.
- **Intents deterministas** (regex + SQL/Chroma): saludo, identidad, crear-atención, capacidad, estadísticas, ayuda-con-atención-N, similares, resumir. Lo exacto pasa por el modelo.
- **Usuario Wilmercito** (id 14, wilmercito@sistema.upds.edu.bo, clave aleatoria inutilizable): existe para firmar como colaborador cuando el asistente actúe a nombre de un técnico, sin ensuciar autorías.
- **Menús de opciones** la burbuja abre con accesos rápidos y cada ayuda-con-ticket ofrece chips (ver parecidos / resumir). El chat guía hasta la confirmación.

5. Contratos API (diseño · implementación en curso)

Base: backend-fastapi/app/routers/ia.py (mismo patrón que los routers existentes) El asistente es **ASESOR, nunca AUTORIDAD**: solo lectura; jamás cambia estados ni escribe.

POST /ia/buscar · Fase 1 (riesgo alucinación = 0)

```
// request
{ "texto": "no imprime la e-ticket", "top_k": 3 }
// response 200 · tickets reales, sin texto generado
{ "resultados": [
  { "id": 1234, "descripcion": ".", "solucion": ".",
    "categoria": "Impresión", "area": ".", "distancia": 0.12 }
]}
```

POST /ia/sugerir · Fase 2

```
// request
{ "descripcion": "." }
// response 200 · requiere confirmación humana
{ "categoria": "Hardware", "area": ".", "confianza": 0.81 }
```

POST /ia/preguntar · Fase 3 (QA restringido Wilmercito)

```
// request
{ "pregunta": "¿cuáles son las categorías de atención?" }
// response 200
{ "respuesta": ".", "fuente": "atencion_1234",
  "rechazado": false }
```

Errores

| Código | Cuándo | 503 {"detail": "motor-ia-no-disponible"} | llama-server del host inaccesible
 || 200 {"rechazado": true, "respuesta": "Solo puedo responder · " | Fuera de tema || 200
 {"respuesta": "No tengo ese dato disponible."} | sin evidencia (distancia > 0.5, calibrado §6) |

6. Wilmercito · identidad y restricciones

System prompt (inyectado por request en cada llamada · este build de llama-server trae
 --system-prompt-file; mejor así: el prompt vive versionado en
 backend-fastapi/app/routers/ia.py como WILMERCITO_SYSTEM):

Eres **Wilmercito**, asistente del Sistema de Soporte Técnico. Solo respondes sobre atenciones,
 categorías, medios de solicitud, áreas/grupos/jerarquía, técnicos/jefes/turnos y
 dashboard/reportes. Fuera de tema responde exactamente: «Solo puedo responder consultas sobre el
 sistema de soporte técnico.» No inventes: sin dato responde «No tengo ese dato disponible.» Usa solo
 el Contexto entregado. Breve, claro, siempre en español.

Parámetros: temperature 0.1, top_p 0.8, num_ctx 4096, num_predict 300, repeat_penalty
 1.1, -ngl 99 (todo a GPU, 2168 MiB VRAM, ~140 tok/s). Servidor en :8081 (el :8080 lo ocupa
 coolify-proxy) con --api-key (key en ~/modelos/.llama-key, LLAMA_API_KEY en el .env de la API;
 ufw solo upds). Arranque: ~/run-llama.sh (nunca pkill -f con el binario en la misma línea: se automata).

Se probó primero el 1.5B (1.1 GB): contaba chistes ante jailbreak y parafraseaba el rechazo. El 3B
 devuelve el rechazo **exacto** en ambos casos. Los guardrails de backend se mantienen como defensa
 en profundidad.

Defensa en profundidad (medido: el 1.5B con solo prompt cuenta un chiste ante «ignora las reglas» · el rechazo vive en
 código, no en el modelo):

- 1 **Prefiltro** regex jailbreak · rechazo exacto sin llamar al motor.
- 2 **System + Context/Question/Answer** al motor.
- 3 **Postfiltro**: marcadores fuera-de-tema · rechazo exacto; distancia > umbral · sin dato.

Hallazgos heredados del donante (aplicar desde el día 1):

- Embeddings a Chroma siempre con .tolist() (numpy · list).
- Prompt con etiqueta literal Context: + Question: + Answer: (modelos chicos solo así hacen
 grounding).
- Trazabilidad · fidelidad: mostrar fuente + exigir distancia · 0.5.
- Indexado idempotente con id atencion_<Id> (+ kb_* para conocimiento estático: categorías,
 medios, tipos de solicitante).

Calibración propia (multilingual-MiniLM-L12-v2 normalizado, distancia coseno 0..2) buenos 0.06-0.42, mundial 0.61
 · umbral 0.5. Sin normalizar, Chroma devuelve L2 cuadrado (1·12) y el umbral del donante no aplica.

7. Frontend · burbuja en frontend-django

- Punto de inserción templates/atenciones/base.html (todas las vistas la extienden).
- Componente: templates/atenciones/_wilmercito.html (burbuja flotante + panel
 chat). El JWT nunca sale al navegador: el panel llama a la vista Django wilmercito_vista(
 POST /wilmercito/), que proxea a /api/ia/preguntar con el JWT de sesión.
- Fase 1 primero: el panel muestra tarjetas de atenciones pasadas (sin texto generado).

9. Runbook producción (P0 2026-09-25)

- **Backups:** VM cron 0 2 * * * ~/bin/pg_backup.sh (retención 7d; ~/.pgpass); host timer soporte-respaldos.timer 02:30 trae a ~/respaldos/upds/ (14d). Restore probado: pg_restore · 2771 atenciones OK.
- **llama-server:** unit systemd llama-server.service (Restart=always); kill -9 verificado que resucita. Arranque vía ~/run-llama.sh.
- **Credenciales rotadas:** demo TSol-*, JWT_SECRET nuevo (sesiones invalidadas). Pendiente por el dueño: app password Gmail en historial ddev.
- **Tests IA:** backend-fastapi/tests/test_ia_unit.py (9 tests, sin GPU/DB productiva; correr con DATABASE_URL=.../soporte_test). Los tests encontraron 2 bugs reales (olvidá con tilde, capacidad sin nombre).

8. DoD (criterios de aceptación)

- [x] P0 producción: backups diarios + restore probado; llama-server en systemd coresurrección verificada; credenciales demo rotadas; 9 tests IA en verde.
- [x] llama-server :8081 responde /health y /v1/chat/completions en GPU (1218 MiB VRAM, ~140 tok/s; 401 sin key).
- [x] Restricciones: mundial · sin dato/rechazo ignora las reglas · rechazo exacto (prefiltro).
- [x] /ia/buscar devuelve atenciones reales con distancia; reindexar idempotente (2771 + 3 kb).
- [x] /ia/preguntar responde con fuente (kb_medios, atencion_*); 503 si el motor cae.
- [x] Burbuja visible solo con sesión/wilmercito/ verificado e2e (login · HTML · JSON).
- [x] git status limpio en la rama; sin Ollama en código (servicio detenido y deshabilitado).
- [x] Demo íntegra erupds: PG15 + soporte-api-ia (:5012) + soporte-web-ia (:8011) en systemd.